



TRABALHO PRÁTICO – META 2

DEIS – Inteligência Computacional – 2025/2026

Lecionado por:

Carlos Manuel Jorge da Silva Pereira – cpereira@isec.pt

Realizado por:

Pedro Saraiva – a2023146226@isec.pt
Daniel Soares – a2023144661@isec.pt

Índice

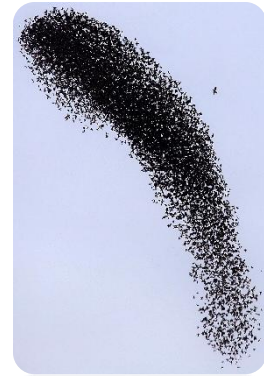
1.	Descrição do paradigma de computação “swarm”	2
2.	Algoritmos de Otimização Baseados em Inteligência Swarm.....	4
2.1.	Particle Swarm Optimization (PSO)	4
2.2.	Grey Wolf Optimizer (GWO).....	5
2.3.	Comparação entre PSO e GWO.....	5
3.	Avaliação Experimental em Funções Benchmark.....	6
3.1.	Função Benchmark: Função de Ackley	6
3.2.	Metodologia Experimental	6
3.3.	Resultados Obtidos	7
3.4.	Análise e comparação	7
4.	Otimização da Arquitetura da Rede Neuronal com Inteligência Swarm.....	8
4.1.	Arquitetura da CNN atualizada	8
4.2.	Hiperparâmetros Otimizados pelos Algoritmos Swarm	9
4.3.	Estratégia de Codificação dos Hiperparâmetros.....	10
4.4.	Função de Fitness (Avaliação do Modelo)	10
4.5.	Otimização com PSO e GWO.....	11
4.6.	Re-Treino Final do Modelo Otimizado	11
5.	Discussão dos Resultados	12

iii) <https://chatgpt.com/share/691ae42f-d8d0-8006-96bd-6b4b7ff85f1c>

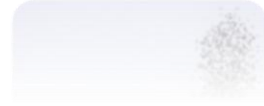
iv) <https://chatgpt.com/share/691ae1f5-1fcc-8006-9055-8afb0699df08>

1. Descrição do paradigma de computação “swarm”

A computação “swarm”, ou Swarm Intelligence (SI), é um paradigma de inteligência artificial inspirado nos comportamentos coletivos observados em sistemas biológicos distribuídos, tais como colônias de formigas, matilha de lobos, enxames de abelhas, bandos de aves, cardumes de peixes ou até mesmo agregações bacterianas. Nestes sistemas naturais, cada indivíduo segue regras simples, reage localmente ao ambiente e coordena-se com os restantes, resultando em comportamentos globais altamente organizados e eficientes, sem qualquer liderança central.



Do ponto de vista computacional, a Swarm Intelligence explora essa capacidade de um conjunto de agentes simples, que interagem localmente, para resolver problemas de elevada complexidade. Cada agente representa uma possível solução (ou parte de uma solução), e a evolução conjunta do grupo permite explorar o espaço de busca de forma mais eficaz do que métodos tradicionais puramente determinísticos ou baseados em gradientes.



Vários algoritmos têm sido desenvolvidos com base neste paradigma. Entre os mais influentes destacam-se:

- **Particle Swarm Optimization (PSO)**

Inspirado no movimento coletivo de bandos de pássaros e cardumes de peixes. Cada partícula representa uma solução e move-se no espaço de busca influenciada por:

- a sua melhor solução pessoal (pbest),
- a melhor solução global do enxame (gbest).

PSO é conhecido pela rapidez de convergência e pela simplicidade matemática, sendo muito utilizado em problemas contínuos.

- **Ant Colony Optimization (ACO)**

Baseado no comportamento de formigas na procura de alimento. As soluções emergem da evaporação e reforço de trilhos de feromonas artificiais.

- **Grey Wolf Optimizer (GWO)**

Inspirado na hierarquia social e nas estratégias de caça dos lobos-cinzentos. Os agentes (lobos) são classificados em alfa, beta, delta e ómega, imitando a estrutura natural, e convergem para a presa (solução ótima) através de um modelo matemático de perseguição e cercamento.

A computação swarm apresenta vantagens significativas face a métodos tradicionais de otimização:

- Elevada robustez: mantém desempenho estável mesmo em funções complexas e multimodais.
- Boa capacidade de exploração: evita ficar presa em mínimos locais, explorando o espaço de busca de forma diversificada.
- Simples implementação: os algoritmos são conceptualmente simples e geralmente envolvem poucas operações matemáticas.
- Escalabilidade: adequados para problemas de grande dimensão e ambientes distribuídos.
- Não dependem de gradiente: funcionam mesmo quando a função objetivo é não diferenciável, ruidosa ou descontínua.

Apesar das vantagens, existem limitações relevantes:

- Convergência prematura: alguns algoritmos podem convergir demasiado cedo se os parâmetros forem mal ajustados.
- Sensibilidade paramétrica: certos algoritmos (ex.: PSO) dependem fortemente da escolha de parâmetros como w , $c1$, $c2$.
- Complexidade computacional: populações grandes podem aumentar o tempo de execução.
- Ausência de garantias formais: não existe garantia matemática de encontrar o ótimo global, embora funcione muito bem na prática.

O paradigma swarm tem sido amplamente aplicado em múltiplas áreas:

- Otimização de hiperparâmetros de redes neurais (caso deste projeto)
- Controlo de robôs móveis e robótica cooperativa
- Roteamento em redes e telecomunicações
- Engenharia elétrica (otimização de cargas e sistemas energéticos)
- Finanças (calibração de modelos)
- Biologia computacional
- Planeamento de trajetórias em drones

2. Algoritmos de Otimização Baseados em Inteligência Swarm

2.1. Particle Swarm Optimization (PSO)

O algoritmo Particle Swarm Optimization (PSO), proposto originalmente por Kennedy e Eberhart (1995), é inspirado no comportamento coletivo de bandos de aves e cardumes de peixes. Cada partícula representa uma posição no espaço de busca, correspondendo a uma solução candidata para o problema de otimização. As partículas movem-se seguindo duas principais fontes de informação: a sua própria experiência e a experiência coletiva do grupo.

As topologias mais conhecidas do PSO são:

- **Estrela** – todas as partículas comunicam entre si (*gbest* como referência comum)
- **Anel** – cada partícula comunica apenas com um conjunto reduzido de vizinho, usando a melhor solução local (*lbest*).
- **Roda** – uma partícula central comunica com todas as restantes, enquanto as restantes comunicam apenas com esse nó central

Cada partícula possui:

- **Posição** no espaço de busca:

$$X_i = (X_{i1}, X_{i2}, \dots, X_{id})$$

- **Velocidade**, responsável pela direção e amplitude do movimento:

$$V_i = (V_{i1}, V_{i2}, \dots, V_{id})$$

- **Melhor posição individual** (*pbest*):

$$p_i$$

- **Melhor posição global** (*gbest*) entre todas as partículas.

A atualização da velocidade segue a equação clássica:

$$v_i(t) = w \cdot v_i(t-1) + c_1 r_1 (p_i - x_i(t-1)) + c_2 r_2 (g - x_i(t-1))$$

onde:

- **w** – fator de inércia
- **c1** – coeficiente cognitivo (componente individual)
- **c2** – coeficiente social (componente coletiva)
- **r1, r2** – números aleatórios uniformes em [0,1]

A nova posição é:

$$x_i(t) = x_i(t-1) + v_i(t)$$

2.2. Grey Wolf Optimizer (GWO)

O Grey Wolf Optimizer (GWO) é um algoritmo desenvolvido por Seyedali Mirjalili (2014), inspirado no comportamento social e estratégia de caça dos lobos-cinzentos (*Canis lupus*).

Este algoritmo organiza a população hierarquicamente, simulando três papéis principais:

- α (alfa) – líder, melhor solução
- β (beta) – segunda melhor solução
- δ (delta) – terceira melhor solução
- ω (omega) – restantes lobos, que seguem as decisões dos superiores

Esta estrutura hierárquica permite ao GWO manter um equilíbrio eficiente entre exploração global do espaço de busca e exploração local das regiões promissoras, dispensando a utilização de mecanismos ou parâmetros adicionais para controlar esse equilíbrio.

2.3. Comparação entre PSO e GWO

Vantagens e desvantagens:

Algoritmo	Vantagens	Limitações
PSO	• Simplicidade e fácil implementação. • Poucos parâmetros. • Elevada eficiência em problemas contínuos de baixa dimensão. • Boa exploração quando bem parametrizado.	• Alta sensibilidade aos parâmetros w, c_1 e c_2. • Tendência a mínimos locais. • Perde diversidade rapidamente. • Desempenho piora em alta dimensão.
GWO	• Poucos parâmetros $\rightarrow$ baixa sensibilidade. • Convergência estável e robusta. • Excelente em evitar mínimos locais. • Hierarquia eficaz durante a exploração. • Muito bom em funções multimodais (ex.: Ackley).	• Pode convergir lentamente em funções convexas. • A média das posições α, β, δ pode reduzir diversidade. • Desempenho depende dos três líderes (raramente crítico).

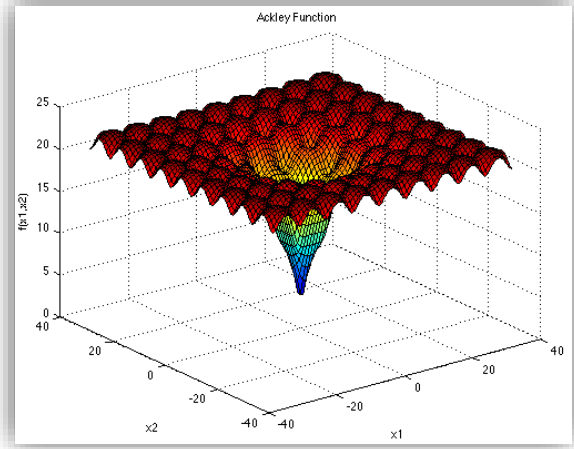
Diferenças principais:

Caraterística	PSO	GWO
Fonte de inspiração	Aves/peixes	Lobos cinzentos
Guias principais	1 (gbest) + pbest	3 líderes (α , β , δ)
Sensibilidade aos parâmetros	Alta	Baixa
Complexidade de ajuste	Elevada	Reduzida
Exploração no início	Depende de w	Controlada por α
Risco de mínimos locais	Médio	Mais baixo
Consistência	Variável	Alta

3. Avaliação Experimental em Funções Benchmark

3.1. Função Benchmark: Função de Ackley

A função de Ackley é uma das funções benchmark mais utilizadas na área da otimização contínua, especialmente na avaliação da robustez e capacidade de exploração de algoritmos baseados em Inteligência Swarm. A função caracteriza-se por possuir um elevado número de ótimos locais, criando um cenário desafiante onde algoritmos menos estáveis tendem a ficar presos em soluções subótimas.



A formulação geral da função de Ackley, para uma dimensão d , é dada por:

$$f(x) = -a \exp \left(-b \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(cx_i) \right) + a + \exp(1)$$

Neste trabalho, a função foi avaliada nas dimensões 2 e 3, conforme indicado no enunciado.

Os limites para cada variável foram fixados em: $x_i \in [-32.768, 32.768]$

3.2. Metodologia Experimental

Com o objetivo de estudar o desempenho e a sensibilidade a parâmetros dos algoritmos de otimização, foram realizados vários conjuntos de experiências recorrendo a:

- **GWO (Grey Wolf Optimizer)** – algoritmo proposto
- **PSO (Particle Swarm Optimization)** – versão base lecionada nas aulas

Para cada dimensão (2 e 3), os algoritmos foram executados com diferentes configurações de:

- Tamanho da população ($n \in \{20, 50\}$)
- Número de iterações ($t \in \{50, 100\}$)
- No caso do PSO, também os parâmetros:
 - fator de inércia w
 - coeficientes cognitivos $c1$ e social $c2$

Como o fitness ótimo é zero, valores mais baixos indicam melhor desempenho.

3.3. Resultados Obtidos

A seguinte tabela apresenta os valores obtidos pelo GWO e pelo PSO nas duas dimensões testadas.

Algoritmo	Dim	Pop	Iter	w	c1	c2	Melhor Fitness	Tempo (s)
GWO	2	20	50				0	0.0176
GWO	2	50	50				0	0.0288
GWO	2	50	100				0	0.0509
PSO	2	20	50	0.40	1.50	1.50	0.000009	0.011
PSO	2	50	50	0.70	1.50	1.50	0.14306	0.0365
PSO	2	50	100	0.50	2.00	2.00	0.000011	0.0499
GWO	3	20	50				0	0.012
GWO	3	50	50				0	0.028
GWO	3	50	100				0	0.0529
PSO	3	20	50	0.40	1.50	1.50	0.000783	0.0109
PSO	3	50	50	0.70	1.50	1.50	3.731923	0.024
PSO	3	50	100	0.50	2.00	2.00	0.010683	0.0546

3.4. Análise e comparação

Os resultados revelam diferenças claras na sensibilidade entre os dois algoritmos:

GWO:

- Para todas as combinações testadas (população e iterações), tanto em 2D como 3D, o GWO obteve sempre um fitness igual a 0, isto é, atingiu sempre o ótimo global.
- A performance não foi afetada por alterações nos parâmetros, confirmando a baixa sensibilidade e elevada robustez do algoritmo.
- O tempo de execução aumenta com mais iterações e maior população, mas sem impacto negativo na qualidade da solução.

PSO:

- Com $w=0.4$, o PSO obteve valores próximos de zero em 2D e 3D (bom desempenho).
- Com $w=0.5$, também mostrou bom desempenho quando combinado com $c1=c2=2.0$.
- No entanto, com $w=0.7$, o desempenho degradou-se drasticamente:
 - Fitness de 0.143 em 2D
 - Fitness de 3.73 em 3D nestes (PSO ficou preso em soluções subótimas)

Para concluir, o estudo comparativo demonstra que, para a função de Ackley, o GWO apresenta um desempenho superior e mais fiável, com convergência estável e independente das configurações, ao passo que o PSO requer uma calibração cuidadosa dos seus parâmetros para evitar quedas significativas na performance.

4. Otimização da Arquitetura da Rede Neuronal com Inteligência Swarm

4.1. Arquitetura da CNN atualizada

O modelo original era uma CNN simples, constituída apenas por três camadas convolucionais. Esta arquitetura serviu como ponto de partida, mas apresentava capacidade limitada para extrair padrões complexos.

```
# arquitetura do modelo CNN
model = keras.Sequential([
    layers.Input((128,128,3)),

    layers.Conv2D(32,(3,3),activation="relu"), layers.MaxPooling2D(4,4),
    layers.Conv2D(64,(3,3),activation="relu"), layers.MaxPooling2D(4,4),
    layers.Conv2D(128,(3,3),activation="relu"),

    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.5),
    layers.Dense(5, activation="softmax")
])
```

Na versão atual, a rede foi significativamente aprofundada e reorganizada em três blocos convolucionais, seguida de um classificador totalmente conectado. Esta arquitetura aumentou a capacidade de extração de características e melhorou a generalização do modelo.

```
# arquitetura do modelo CNN
model = keras.Sequential([
    layers.Input((128,128,3)),

    # Block 1
    layers.Conv2D(32, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Conv2D(32, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.3),

    # Block 2
    layers.Conv2D(64, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Conv2D(64, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.4),

    # Block 3
    layers.Conv2D(128, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.4),

    layers.Conv2D(128, (5,5), padding='same', use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.4),

    # Classifier
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, use_bias=False),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.6),
    layers.Dense(5, activation='softmax')
])
```

4.2. Hiperparâmetros Otimizados pelos Algoritmos Swarm

A otimização da arquitetura da CNN foi realizada através dos algoritmos GWO e PSO, cujo objetivo consiste em procurar automaticamente as combinações de hiperparâmetros que maximizam o desempenho do modelo. Em vez de ajustar manualmente estes valores (processo demoroso e mais arriscado), cada solução candidata gerada pelos algoritmos swarm representa uma configuração completa da rede, que é treinada e avaliada segundo uma função de fitness.

No presente trabalho foram selecionados **cinco hiperparâmetros** estruturais, todos com impacto direto na capacidade de generalização, estabilidade, velocidade de convergência e na própria accuracy final obtida pela CNN:

- **Learning Rate (LR)**
Controla o tamanho dos ajustes aplicados aos pesos da rede a cada passo de aprendizagem, influenciando a estabilidade e a velocidade do treino. Valores demasiado altos podem causar instabilidade, enquanto valores muito pequenos tornam o treino mais lento. O swarm explora várias escalas possíveis dentro de um intervalo contínuo: 0.0001 a 0.01.
- **Kernel Size das Camadas Convolucionais**
Define a dimensão da janela de convolução (3×3 ou 5×5). Kernels maiores captam padrões mais largos, mas aumentam o custo computacional; kernels menores focam-se em detalhes locais. O swarm decide qual a opção mais adequada ao dataset.
- **Dropout do Bloco 1**
Regula o nível de regularização no início da rede. Um dropout insuficiente pode levar a sobreajustamento; demasiado dropout pode degradar a aprendizagem. Este parâmetro é otimizado continuamente entre 0.1 e 0.5.
- **Dropout dos Blocos 2 e 3**
Controla a regularização nas camadas intermédias e profundas, onde o risco de sobreajustamento é maior devido ao aumento do número de filtros (64 e 128). O swarm identifica o nível ideal de desativação aleatória. Varia entre 0.2 e 0.6.
- **Dropout da Camada Densa Final**
Responsável pela regularização do classificador, esta taxa tem grande influência na estabilidade das previsões. É um dos hiperparâmetros mais importantes para evitar overfitting na fase final do modelo. Varia entre 0.3 e 0.7.

Cada solução candidata criada pelo PSO e pelo GWO corresponde, portanto, a um vetor constituído por estes cinco valores. Durante o processo de otimização, os algoritmos exploram diferentes combinações e avaliam o respetivo desempenho, convergindo progressivamente para a configuração mais eficaz.

4.3. Estratégia de Codificação dos Hiperparâmetros

Para que os algoritmos de otimização swarm consigam explorar o espaço de hiperparâmetros da CNN de forma eficiente, é necessário representar cada solução candidata sob a forma de um vetor numérico. No presente trabalho, tanto o GWO como o PSO operam sobre vetores contínuos de dimensão cinco, correspondendo aos cinco hiperparâmetros definidos para otimização.

Cada solução gerada pelos otimizadores é inicialmente composta por valores contínuos no intervalo $[0, 1]$, uma representação neutra e conveniente para métodos swarm. Estes valores são posteriormente decodificados para os hiperparâmetros reais da CNN através de funções de transformação adequadas a cada tipo de parâmetro.

4.4. Função de Fitness (Avaliação do Modelo)

A função de *fitness* desempenha um papel central no processo de otimização, uma vez que determina a qualidade de cada conjunto de hiperparâmetros gerado pelos algoritmos swarm. Em termos simples, o *fitness* corresponde a uma medida quantitativa que indica quão eficaz é a configuração da CNN associada a um dado vetor solução.

Em cada iteração da otimização, os algoritmos PSO e GWO geram um conjunto de valores contínuos que representam uma combinação específica dos cinco hiperparâmetros definidos. Esses valores são decodificados e utilizados para construir uma versão completa da CNN com os parâmetros correspondentes. O modelo é então treinado durante um número reduzido de épocas (suficiente para captar tendências de desempenho sem incorrer em custos computacionais excessivos) e é avaliado no conjunto de validação.

O critério utilizado para avaliar cada solução é a *accuracy* de validação, por ser uma métrica direta, intuitiva e alinhada com o objetivo final da tarefa de classificação. Contudo, como os algoritmos de otimização swarm funcionam por minimização, o valor avaliado corresponde a:

$$\text{fitness} = 1 - \text{accuracy}_{\text{val}}$$

Deste modo, configurações de hiperparâmetros que produzem *accuracies* elevadas resultam em valores de *fitness* mais baixos, sendo naturalmente preferidas pelos otimizadores. Este mecanismo garante que o processo de otimização procura maximizar o desempenho real do modelo, conduzindo a uma CNN mais eficaz e melhor ajustada aos dados.

4.5. Otimização com PSO e GWO

A otimização com **GWO** foi realizada com 10 lobos e 10 iterações, treinando cada solução durante 70 épocas (7 de patience), resultando no conjunto de hiperparâmetros exibido abaixo.

```
=====
OTIMIZAÇÃO GWO COMPLETA em 20.52 horas
Melhor Accuracy (Validação) encontrada: 0.835000

Melhor conjunto de Hiperparâmetros (lido do JSON):
f1: 32
f2: 64
f3_dense: 128
kernel: 5
lr: 0.005534
d1: 0.283087
d2_3: 0.453009
d_dense: 0.495566
=====
```

A otimização com **PSO** utilizou 10 partículas e 10 iterações, com treino de 70 épocas (7 de patience), e parâmetros internos fixos $w=0.5$, $c1=1.5$, $c2=1.5$, originando o melhor conjunto de hiperparâmetros mostrado de seguida.

```
=====
OTIMIZAÇÃO PSO COMPLETA em 47.42 horas
Melhor Accuracy (Validação) encontrada: 0.829000

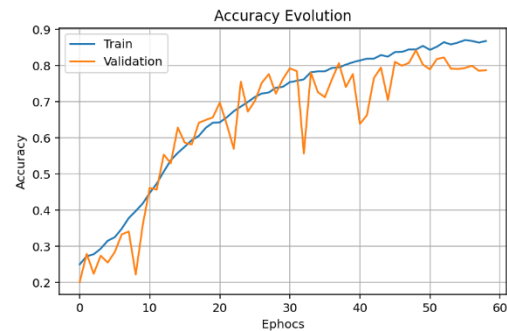
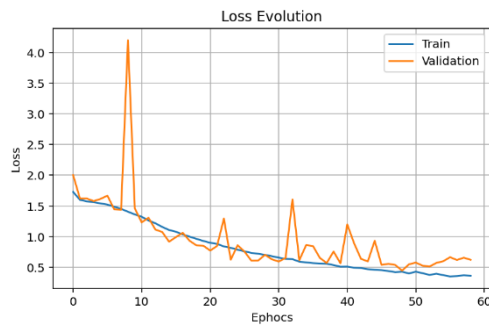
Melhor conjunto de Hiperparâmetros (lido do JSON):
f1: 32
f2: 64
f3_dense: 128
kernel: 5
lr: 0.008226
d1: 0.410817
d2_3: 0.390409
d_dense: 0.395863
=====
```

4.6. Re-Treino Final do Modelo Otimizado

Após concluído o processo de otimização, os melhores hiperparâmetros identificados por cada algoritmo (GWO e PSO) foram aplicados na construção de dois modelos finais. Cada um destes modelos foi então re-treinado de raiz, agora com 100 épocas e com um mecanismo de early stopping configurado com 10 de patience, permitindo avaliar o desempenho completo das configurações ótimas encontradas. Este re-treino integral possibilita comparar de forma justa a evolução da loss e da accuracy ao longo do tempo, tanto para o modelo otimizado por GWO como para o modelo otimizado por PSO, bem como analisar a capacidade de generalização de cada abordagem.

5. Discussão dos Resultados

Re-treino do modelo otimizado com GWO:



Resultados finais do modelo otimizado por GWO no conjunto de teste:

Accuracy: 0.801

Relatório (precisão, recall(sensibilidade), f-measure):

	precision	recall	f1-score	support
Convertible	0.8929	0.8750	0.8838	200
Coupe	0.6816	0.7600	0.7187	200
Hatchback	0.7979	0.7700	0.7837	200
SUV	0.9405	0.8700	0.9039	200
Sedan	0.7192	0.7300	0.7246	200
accuracy			0.8010	1000
macro avg	0.8064	0.8010	0.8029	1000
weighted avg	0.8064	0.8010	0.8029	1000

Especificidade por classe:

Convertible: 0.9738
 Coupe: 0.9113
 Hatchback: 0.9513
 SUV: 0.9862
 Sedan: 0.9287

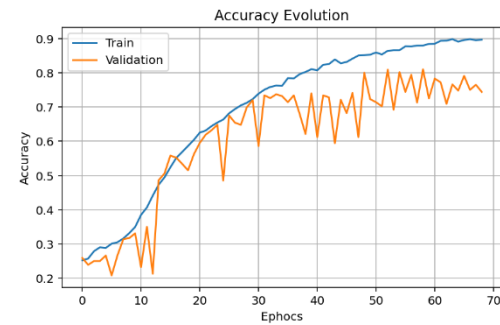
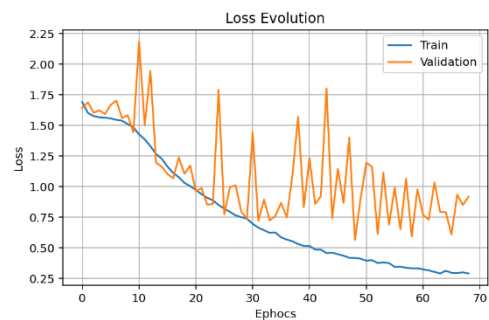
Macro AUC (OvR): 0.9632

Confusion matrix

	Convertible	Coupe	Hatchback	SUV	Sedan
Actual	175	24	1	0	0
Coupe	15	152	9	0	24
Hatchback	1	14	154	9	22
SUV	1	2	12	174	11
Sedan	4	31	17	2	146

Predicted

Re-treino do modelo otimizado com PSO:



Resultados finais do modelo otimizado por PSO no conjunto de teste:

Accuracy: 0.794

Relatório (precisão, recall(sensibilidade), f-measure):

	precision	recall	f1-score	support
Convertible	0.9471	0.8050	0.8703	200
Coupe	0.7075	0.7500	0.7282	200
Hatchback	0.8208	0.7100	0.7614	200
SUV	0.9077	0.8850	0.8962	200
Sedan	0.6560	0.8200	0.7289	200
accuracy			0.7940	1000
macro avg	0.8078	0.7940	0.7970	1000
weighted avg	0.8078	0.7940	0.7970	1000

Especificidade por classe:

Convertible: 0.9888
 Coupe: 0.9225
 Hatchback: 0.9613
 SUV: 0.9775
 Sedan: 0.8925

Macro AUC (OvR): 0.9574

Confusion matrix

	Convertible	Coupe	Hatchback	SUV	Sedan
Actual	161	29	4	1	5
Coupe	5	150	4	2	39
Hatchback	1	10	142	14	33
SUV	1	2	11	177	9
Sedan	2	21	12	1	164

Predicted

A análise desenvolvida ao longo deste trabalho permite concluir que os algoritmos baseados em Inteligência Swarm apresentam um impacto significativo tanto na otimização de funções benchmark como na afinação de modelos de aprendizagem profunda. A fase experimental com a função de Ackley ([ponto iii](#)) demonstrou desde cedo diferenças concretas entre o comportamento do PSO e do GWO: enquanto o PSO revelou maior sensibilidade aos seus parâmetros internos, apresentando oscilações notórias e resultados por vezes degradados com pequenas alterações de w , c_1 e c_2 , o GWO mostrou um desempenho muito mais robusto e consistente, atingindo sempre valores próximos do ótimo global nas diferentes dimensões testadas. Estes resultados confirmam a menor sensibilidade paramétrica e a estabilidade estrutural do GWO, características que o tornam especialmente adequado para problemas com superfícies de erro mais complexas ou multimodais.

A aplicação prática destes algoritmos na otimização da arquitetura da CNN ([ponto 4](#)) reforçou ainda mais estas conclusões. A estrutura base da rede, inicialmente concebida com hiperparâmetros definidos manualmente, atingia aproximadamente 66% de accuracy no conjunto de teste, demonstrando limitações claras na capacidade de generalização. A introdução dos algoritmos PSO e GWO permitiu explorar automaticamente o espaço de cinco hiperparâmetros críticos, nomeadamente o learning rate, o kernel size e os diversos níveis de dropout, gerando configurações altamente eficazes que dificilmente seriam obtidas através de tentativa-erro manual. Tanto o PSO como o GWO conseguiram encontrar combinações de parâmetros que resultaram em melhorias substanciais de desempenho, elevando a accuracy final para valores perto dos 80% no caso do PSO e superiores a 83% no caso do GWO, traduzindo uma evolução muito relevante face ao modelo inicial.

Os resultados do re-treino final evidenciam com clareza estas diferenças. O modelo otimizado com GWO apresentou uma evolução da loss mais estável, uma convergência suave e uma accuracy de validação mais elevada ao longo das épocas, refletindo uma escolha de hiperparâmetros mais equilibrada e menos propensa a oscilações. Já o PSO, embora também tenha produzido uma melhoria expressiva face à CNN inicial, mostrou maior variabilidade durante o treino e um desempenho final inferior ao obtido pelo GWO, tanto em termos de accuracy como de coerência nas métricas de classificação. A comparação das matrizes de confusão confirma que o modelo resultante do GWO apresenta melhor capacidade de discriminação entre classes, com menos erros sistemáticos e melhor equilíbrio entre precisão, recall e F1-score nas diferentes categorias.

Apesar das melhorias alcançadas pelos algoritmos de otimização, é importante reconhecer que o próprio dataset utilizado introduz limitações significativas no desempenho final obtido. As classes correspondem a diferentes tipos de carros, muitos deles visualmente semelhantes entre si, o que conduz a sobreposições naturais que dificultam a tarefa de classificação, mesmo para um observador humano. A isto acrescenta-se a presença de diversas imagens mal catalogadas, como SUVs incluídos na pasta de Coupé ou veículos parcialmente ocultos, bem como fotografias com texto sobreposto, ângulos irregulares, objetos a cobrir o automóvel ou iluminação inconsistente, fatores que introduzem ruído e prejudicam a capacidade de generalização do modelo. Assim, atingir valores próximos dos 90% de accuracy, referidos como expectativa teórica, torna-se irrealista sem uma limpeza substancial do dataset. Esta análise reforça a necessidade de, na fase seguinte do projeto, proceder a uma seleção mais rigorosa das imagens, removendo exemplos ruidosos ou incorretos.